

Exact?

Local Search

Global Search

Overview

Related Tactics

Other ITPs

History

How ‘exact?’ Works

Discrimination Trees and SolveByElim in Lean 4

Evgenia Karunus

August 3, 2026

Exact?

Local Search

Global Search

Overview

Related Tactics

Other ITPs

History

Demo

Exact?

Local Search

Global Search

Overview

Related Tactics

Other ITPs

History

Exact?

```
example (l1 l2 l3 : List Nat) (h1 : l1.length ≤ l2.length) (h2 : l3 = l2) :  
l1.length ≤ l3.length := by  
  exact?
```

Exact?

Local Search

Global Search

Overview

Related Tactics

Other ITPs

History

Exact?

```
example (l1 l2 l3 : List Nat) (h1 : l1.length ≤ l2.length) (h2 : l3 = l2) :  
l1.length ≤ l3.length := by  
  exact le_of_le_of_eq h1 (congrArg List.length (id (Eq.symm h2)))
```

Exact?

Local Search

Global Search

Overview

Related Tactics

Other ITPs

History

Exact?

```
example (l1 l2 l3 : List Nat) (h1 : l1.length ≤ l2.length) (h2 : l3 = l2) :  
  l1.length ≤ l3.length := by  
  exact le_of_le_of_eq h1 (congrArg List.length (id (Eq.symm h2)))
```

Exact? works by

- finding ONE lemma from the list of ALL lemmas ever (uses `DiscrTree`)
- dealing with the remaining goals using LOCAL hypotheses (uses `SolveByElim`)

Exact?

Local Search

Global Search

Overview

Related Tactics

Other ITPs

History

Exact?

```
example (l1 l2 l3 : List Nat) (h1 : l1.length ≤ l2.length) (h2 : l3 = l2) :  
  l1.length ≤ l3.length := by  
  exact le_of_le_of_eq h1 (congrArg List.length (id (Eq.symm h2)))
```

Exact? works by

- finding ONE lemma from the list of ALL lemmas ever (uses **DiscrTree**)
- dealing with the remaining goals using LOCAL hypotheses (uses **SolveByElim**)

exact	<u>le_of_le_of_eq</u>	<u>h1</u>	<u>(congrArg List.length (id (Eq.symm h2)))</u>
	Global Search (DiscrTree)	Local Search (SolveByElim)	Local Search (SolveByElim)

Option

exact? +grind/+try?

exact? using h₁, h₂

Effect

use **grind** / **try?** as fallback discharger for subgoals

"provides identifiers in the local context that must be used"

Exact?

Local Search

- SolveByElim
- Implementation
- Lemma List
- Example

Global Search

Overview

Related Tactics

Other ITPs

History

Local Search

`exact` `le_of_le_of_eq` `h1` `(congrArg List.length (id (Eq.symm h2)))`

Global Search Local Search Local Search
(DiscrTree) (SolveByElim) (SolveByElim)

SolveByElim

Exact?

Local Search

- SolveByElim
- Implementation
- Lemma List
- Example

Global Search

Overview

Related Tactics

Other ITPs

History

The `solve_by_elim` tactic has been ported from *Std* [aka Batteries] to *Lean* so that library search can use it.

Lean v4.7.0 release notes



Exact?

Local Search

- SolveByElim
- Implementation
- Lemma List
- Example

Global Search

Overview

Related Tactics

Other ITPs

History

SolveByElim

The `solve_by_elim` tactic has been ported from *Std* [aka Batteries] to *Lean* so that library search can use it.

Lean v4.7.0 release notes



`solve_by_elim` as a tactic:

- Mathlib: 49 times
- Lean core: 8 times
- Batteries: 2 times

SolveByElim

Exact?

Local Search

- SolveByElim
- Implementation
- Lemma List
- Example

Global Search

Overview

Related Tactics

Other ITPs

History

The `solve_by_elim` tactic has been ported from *Std* [aka Batteries] to *Lean* so that library search can use it.

Lean v4.7.0 release notes

`solve_by_elim` as a tactic:

- Mathlib: 49 times
- Lean core: 8 times
- Batteries: 2 times

`solve_by_elim` (and `try?`) within tactic implementations:

- Mathlib: 10 tactics
`tauto`, `mono`, `interval_cases`, `linarith`, `linear_combination`, `positivity`
- Lean core: 3 tactics
`apply?`, `exact?`, `rw?`
- Batteries: 0 tactics

SolveByElim: Implementation

Exact?

Local Search

- SolveByElim
- Implementation
- Lemma List
- Example

Global Search

Overview

Related Tactics

Other ITPs

History

SolveByElim "eliminates" the current goal by running [apply, apply, apply...]

```
lemma list: [h1 : R → Q, h2 : P → Q, h3 : P]
```

```
goal: ⊢ Q
```

Exact?

Local Search

- SolveByElim
- Implementation
- Lemma List
- Example

Global Search

Overview

Related Tactics

Other ITPs

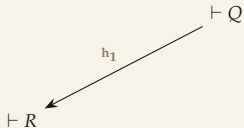
History

SolveByElim: Implementation

SolveByElim "eliminates" the current goal by running [apply, apply, apply...]

```
lemma list: [h1 : R → Q, h2 : P → Q, h3 : P]
```

```
goal: ⊢ Q
```



SolveByElim: Implementation

Exact?

Local Search

- SolveByElim
- Implementation
- Lemma List
- Example

Global Search

Overview

Related Tactics

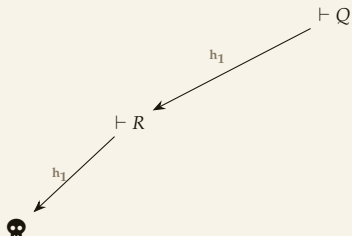
Other ITPs

History

SolveByElim "eliminates" the current goal by running [apply, apply, apply...]

```
lemma list: [h1 : R → Q, h2 : P → Q, h3 : P]
```

```
goal: ⊢ Q
```



Exact?

Local Search

- SolveByElim
- Implementation
- Lemma List
- Example

Global Search

Overview

Related Tactics

Other ITPs

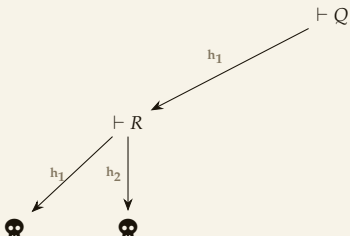
History

SolveByElim: Implementation

SolveByElim "eliminates" the current goal by running [apply, apply, apply...]

lemma list: $[h_1 : R \rightarrow Q, h_2 : P \rightarrow Q, h_3 : P]$

goal: $\vdash Q$



SolveByElim: Implementation

Exact?

Local Search

- SolveByElim
- Implementation
- Lemma List
- Example

Global Search

Overview

Related Tactics

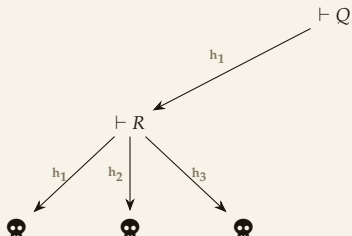
Other ITPs

History

SolveByElim "eliminates" the current goal by running [apply, apply, apply...]

lemma list: $[h_1 : R \rightarrow Q, h_2 : P \rightarrow Q, h_3 : P]$

goal: $\vdash Q$

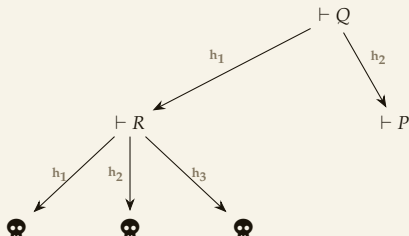


SolveByElim: Implementation

SolveByElim "eliminates" the current goal by running [apply, apply, apply...]

lemma list: $[h_1 : R \rightarrow Q, h_2 : P \rightarrow Q, h_3 : P]$

goal: $\vdash Q$



SolveByElim: Implementation

Exact?

Local Search

- SolveByElim
- Implementation
- Lemma List
- Example

Global Search

Overview

Related Tactics

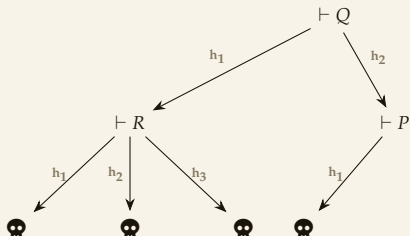
Other ITPs

History

SolveByElim "eliminates" the current goal by running [apply, apply, apply...]

lemma list: $[h_1 : R \rightarrow Q, h_2 : P \rightarrow Q, h_3 : P]$

goal: $\vdash Q$



SolveByElim: Implementation

Exact?

Local Search

- SolveByElim
- Implementation
- Lemma List
- Example

Global Search

Overview

Related Tactics

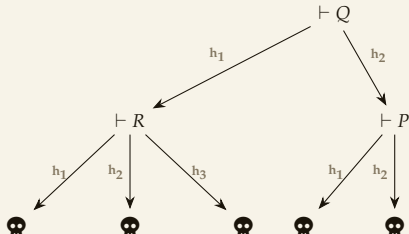
Other ITPs

History

SolveByElim "eliminates" the current goal by running [apply, apply, apply...]

lemma list: $[h_1 : R \rightarrow Q, h_2 : P \rightarrow Q, h_3 : P]$

goal: $\vdash Q$



Exact?

Local Search

- SolveByElim
- Implementation
- Lemma List
- Example

Global Search

Overview

Related Tactics

Other ITPs

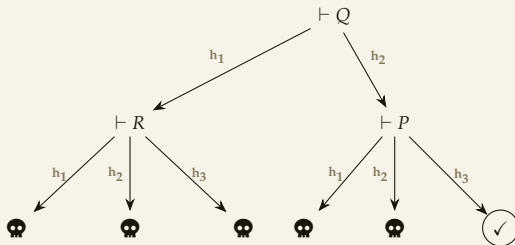
History

SolveByElim: Implementation

SolveByElim "eliminates" the current goal by running [apply, apply, apply...]

lemma list: $[h_1 : R \rightarrow Q, h_2 : P \rightarrow Q, h_3 : P]$

goal: $\vdash Q$



Exact?

Local Search

- SolveByElim
- Implementation
- Lemma List
- Example

Global Search

Overview

Related Tactics

Other ITPs

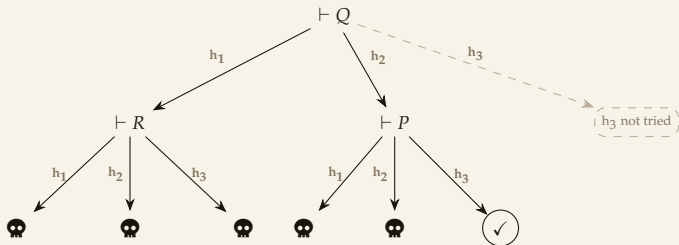
History

SolveByElim: Implementation

SolveByElim "eliminates" the current goal by running [apply, apply, apply...]

lemma list: $[h_1 : R \rightarrow Q, h_2 : P \rightarrow Q, h_3 : P]$

goal: $\vdash Q$



Exact?

Local Search

- SolveByElim
- Implementation
- Lemma List
- Example

Global Search

Overview

Related Tactics

Other ITPs

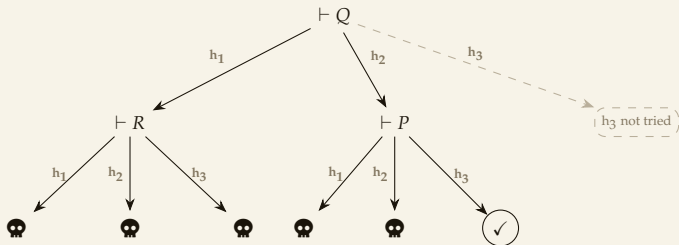
History

SolveByElim: Implementation

SolveByElim "eliminates" the current goal by running [apply, apply, apply...]

lemma list: [h₁ : R → Q, h₂ : P → Q, h₃ : P]

goal: ⊢ Q



Maximum backtracking depth: 6.

Only successful application consumes depth coins.

Will find : h₅ (h₄ (h₃ (h₂ h₁)))

Won't find : h₆ (h₅ (h₄ (h₃ (h₂ h₁))))

Exact?

Local Search

- SolveByElim
- Implementation
- Lemma List
- Example

Global Search

Overview

Related Tactics

Other ITPs

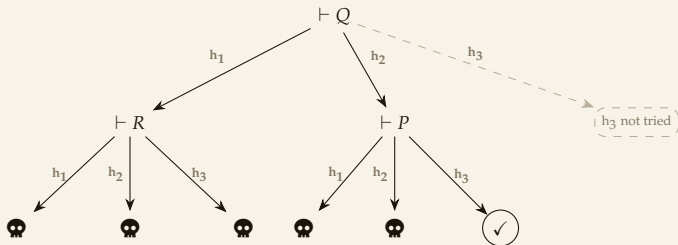
History

SolveByElim: Implementation

SolveByElim "eliminates" the current goal by running [apply, apply, apply...]

lemma list: $[h_1 : R \rightarrow Q, h_2 : P \rightarrow Q, h_3 : P]$

goal: $\vdash Q$



Try at each goal: $[h_1, h_2, h_3]$

Exact?

Local Search

- SolveByElim
- Implementation
- Lemma List
- Example

Global Search

Overview

Related Tactics

Other ITPs

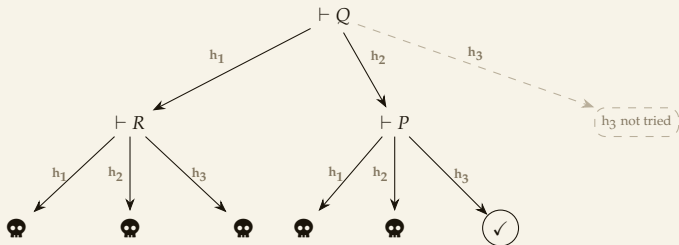
History

SolveByElim: Implementation

SolveByElim "eliminates" the current goal by running [apply, apply, apply...]

lemma list: $[h_1 : R \rightarrow Q, h_2 : P \rightarrow Q, h_3 : P]$

goal: $\vdash Q$



Try at each goal: $[h_1, h_2, h_3, \text{constructor}, \text{intros}, \text{cfg.discharge}]$

Exact?

Local Search

- SolveByElim
- Implementation
- Lemma List
- Example

Global Search

Overview

Related Tactics

Other ITPs

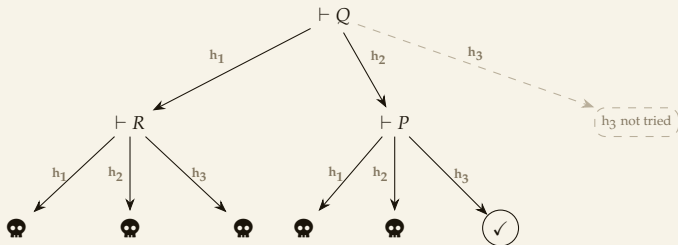
History

SolveByElim: Implementation

SolveByElim "eliminates" the current goal by running [apply, apply, apply...]

lemma list: $[h_1 : R \rightarrow Q, h_2 : P \rightarrow Q, h_3 : P]$

goal: $\vdash Q$



Try at each goal: $[h_1, h_2, h_3, \text{constructor}, \text{intros}, \text{cfg.discharge}]$

constructor has been observed to significantly slow down **exact?** in Mathlib.

/Meta/Tactic/LibrarySearch.lean (Lean v4.29.0 source code)

Exact?

Local Search

- SolveByElim
- Implementation
- Lemma List
- Example

Global Search

Overview

Related Tactics

Other ITPs

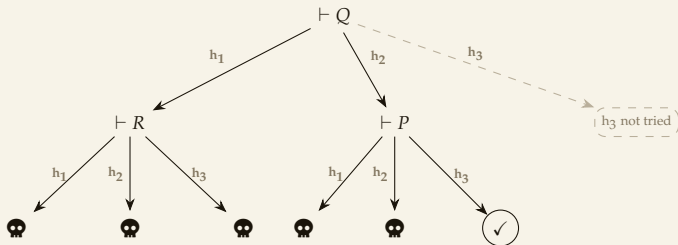
History

SolveByElim: Implementation

SolveByElim "eliminates" the current goal by running [apply, apply, apply...]

lemma list: [h₁ : R → Q, h₂ : P → Q, h₃ : P]

goal: ⊢ Q



Try at each goal: [h₁, h₂, h₃, ~~constructor~~, intros, cfg.discharge]

exact? +grind

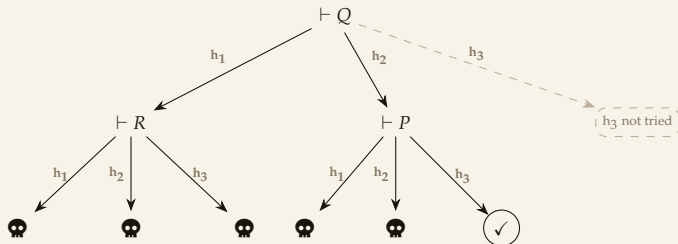
exact? +try?

SolveByElim: Implementation

SolveByElim "eliminates" the current goal by running [apply, apply, apply...]

lemma list: $[h_1 : R \rightarrow Q, h_2 : P \rightarrow Q, h_3 : P]$

goal: $\vdash Q$



Try at each goal: `[lemma_list, constructor, intros, cfg.discharge]`

- `[lemma_list, ..., ...]` - normal backtracking
- `[..., constructor, intros, ..., ...]` - doesn't try intros&discharger if child fails
- `[..., ..., cfg.discharge]` - just exits the tree on fail

Exact?

Local Search

- SolveByElim
- Implementation
- Lemma List
- Example

Global Search

Overview

Related Tactics

Other ITPs

History

SolveByElim: Lemma List

Default `solve_by_elim` **lemma list:**

► **4 hardcoded functions**

```
[  
  rfl : a = a,  
  trivial : True,  
  congrArg : (a = b) (f :  $\alpha \rightarrow \beta$ )  $\rightarrow$  f a = f b,  
  congrFun : (f = g) (a :  $\alpha$ )  $\rightarrow$  f a = g a  
]
```

► **local hypotheses AND their symmetric versions**

```
[  
  ...all hypotheses above the  $\vdash$  in the tactic state,  
  ...their symmetric versions  
]
```

Exact?

Local Search

- SolveByElim
- Implementation
- Lemma List
- Example

Global Search

Overview

Related Tactics

Other ITPs

History

SolveByElim: Lemma List

Options of the `solve_by_elim` tactic:

Option Used	Lemma List
<code>solve_by_elim [t₁, t₂]</code>	default lemma list + <code>[t₁, t₂]</code>
<code>solve_by_elim only [t₁, t₂]</code>	<code>[t₁, t₂]</code>
<code>solve_by_elim using [a₁, a₂]</code>	default lemma list + all lemmas with attrs <code>[a₁, a₂]</code>

Options of the `exact?` tactic:

Option Used	Lemma List
<code>exact? using [h₁, h₂]</code>	FILTER the solutions by the presence of <code>[h₁, h₂]</code>

SolveByElim: Real Example

Exact?

Local Search

- SolveByElim
- Implementation
- Lemma List
- Example

Global Search

Overview

Related Tactics

Other ITPs

History

```
set_option trace.Meta.Tactic.solveByElim true in
example (f : Nat → Nat) (g : Nat → Nat) (h : g = f) : f 42 = g 42 := by
  solve_by_elim
```

SolveByElim: Real Example

Exact?

Local Search

- SolveByElim
- Implementation
- Lemma List
- Example

Global Search

Overview

Related Tactics

Other ITPs

History

```
set_option trace.Meta.Tactic.solveByElim true in
example (f : Nat → Nat) (g : Nat → Nat) (h : g = f) : f 42 = g 42 := by
  solve_by_elim
```

```
lemma list: [
  rfl : a = a,
  trivial : True,
  congrArg : (a = b) (f :  $\alpha \rightarrow \beta$ )  $\rightarrow$  f a = f b,
  congrFun : (f = g) (a :  $\alpha$ )  $\rightarrow$  f a = g a,
  f : Nat  $\rightarrow$  Nat,
  g : Nat  $\rightarrow$  Nat,
  h : g = f,
  id (Eq.symm h) : f = g
]
```

Exact?

Local Search

- SolveByElim
- Implementation
- Lemma List
- Example

Global Search

Overview

Related Tactics

Other ITPs

History

SolveByElim: Real Example

```
set_option trace.Meta.Tactic.solveByElim true in
example (f : Nat → Nat) (g : Nat → Nat) (h : g = f) : f 42 = g 42 := by
  solve_by_elim
```

● working on:

```
f g : Nat → Nat
h : g = f
⊢ f 42 = g 42
```

- ✗ trying to apply: f
- ✗ trying to apply: g
- ✗ trying to apply: h
- ✗ trying to apply: id (Eq.symm h)
- ✗ trying to apply: rfl
- ✗ trying to apply: trivial
- ✓ trying to apply: congrFun

● working on:

```
f g : Nat → Nat
h : g = f
⊢ f = g
```

- ✗ trying to apply: f
- ✗ trying to apply: g
- ✗ trying to apply: h
- ✓ trying to apply: id (Eq.symm h)
- ✓ success: exact congrFun (id (Eq.symm h)) 42

Local Search

Exact?

Local Search

- SolveByElim
- Implementation
- Lemma List
- Example

Global Search

Overview

Related Tactics

Other ITPs

History

`exact` `le_of_le_of_eq` `h1` `(congrArg List.length (id (Eq.symm h2)))`

Global Search Local Search Local Search
(DiscrTree) (SolveByElim) (SolveByElim)

Global Search

Exact?

Local Search

Global Search

- Linear Search
- Indexing
- Example

Overview

Related Tactics

Other ITPs

History

`exact` `le_of_le_of_eq` `h1` `(congrArg List.length (id (Eq.symm h2)))`

Global Search Local Search Local Search

(DiscrTree) (SolveByElim) (SolveByElim)

Exact?

Local Search

Global Search

- Linear Search
- Indexing
- Example

Overview

Related Tactics

Other ITPs

History

Global Search

<code>exact</code>	<code>le_of_le_of_eq</code>	<code>h₁</code>	<code>(congrArg List.length (id (Eq.symm h₂)))</code>
	$\underbrace{\hspace{1.5cm}}$	$\underbrace{\hspace{1.5cm}}$	$\underbrace{\hspace{3.5cm}}$
	Global Search (<u>DiscrTree</u>)	Local Search (SolveByElim)	Local Search (SolveByElim)

What we're proving:

```
example (l1 l2 l3 : List Nat) (h1 : l1.length ≤ l2.length) (h2 : l3 = l2) :  
  l1.length ≤ l3.length := by  
  exact?
```

Our mathlib:

<code>add_2_2</code>	<code>:</code>	<code>2 + 2 = 4</code>
<code>add_2_666</code>	<code>:</code>	<code>2 + 666 = 668</code>
<code>add_3_5</code>	<code>:</code>	<code>3 + 5 = 8</code>
<code>le_of_le_of_eq (h₁ : a ≤ b) (h₂ : b = c)</code>	<code>:</code>	<code>a ≤ c</code>
<code>add_3_7</code>	<code>:</code>	<code>3 + 7 = 10</code>

Exact?

Local Search

Global Search

- Linear Search
- Indexing
- Example

Overview

Related Tactics

Other ITPs

History

Global Search: Linear Search

<code>exact</code>	<code>le_of_le_of_eq</code>	<code>h₁</code>	<code>(congrArg List.length (id (Eq.symm h₂)))</code>
	<u>Global Search</u>	Local Search	Local Search
	<u>(DiscrTree)</u>	(SolveByElim)	(SolveByElim)

What we're proving:

```
example (l1 l2 l3 : List Nat) (h1 : l1.length ≤ l2.length) (h2 : l3 = l2) :  
  l1.length ≤ l3.length := by  
  exact?
```

Our mathlib:

<code>add_2_2</code>	<code>:</code>	<code>2 + 2 = 4</code>	
<code>add_2_666</code>	<code>:</code>	<code>2 + 666 = 668</code>	
<code>add_3_5</code>	<code>:</code>	<code>3 + 5 = 8</code>	
<code>le_of_le_of_eq (h₁ : a ≤ b) (h₂ : b = c)</code>	<code>:</code>	<code>a ≤ c</code>	
<code>add_3_7</code>	<code>:</code>	<code>3 + 7 = 10</code>	

Exact?

Local Search

Global Search

- Linear Search
- Indexing
- Example

Overview

Related Tactics

Other ITPs

History

Global Search: Linear Search

<code>exact</code>	<code>le_of_le_of_eq</code>	<code>h₁</code>	<code>(congrArg List.length (id (Eq.symm h₂)))</code>
	<u>Global Search</u>	Local Search	Local Search
	(DiscrTree)	(SolveByElim)	(SolveByElim)

What we're proving:

```
example (l1 l2 l3 : List Nat) (h1 : l1.length ≤ l2.length) (h2 : l3 = l2) :  
  l1.length ≤ l3.length := by  
  exact?
```

Our mathlib:

<code>add_2_2</code>	<code>:</code>	<code>2 + 2 = 4</code>	✗
<code>add_2_666</code>	<code>:</code>	<code>2 + 666 = 668</code>	✗
<code>add_3_5</code>	<code>:</code>	<code>3 + 5 = 8</code>	
<code>le_of_le_of_eq (h₁ : a ≤ b) (h₂ : b = c)</code>	<code>:</code>	<code>a ≤ c</code>	
<code>add_3_7</code>	<code>:</code>	<code>3 + 7 = 10</code>	

Exact?

Local Search

Global Search

- Linear Search
- Indexing
- Example

Overview

Related Tactics

Other ITPs

History

Global Search: Linear Search

<code>exact</code>	<code>le_of_le_of_eq</code>	<code>h₁</code>	<code>(congrArg List.length (id (Eq.symm h₂)))</code>
	<u>Global Search</u>	Local Search	Local Search
	<u>(DiscrTree)</u>	(SolveByElim)	(SolveByElim)

What we're proving:

```
example (l1 l2 l3 : List Nat) (h1 : l1.length ≤ l2.length) (h2 : l3 = l2) :  
  l1.length ≤ l3.length := by  
  exact?
```

Our mathlib:

<code>add_2_2</code>	<code>:</code>	<code>2 + 2 = 4</code>	✗
<code>add_2_666</code>	<code>:</code>	<code>2 + 666 = 668</code>	✗
<code>add_3_5</code>	<code>:</code>	<code>3 + 5 = 8</code>	✗
<code>le_of_le_of_eq (h₁ : a ≤ b) (h₂ : b = c)</code>	<code>:</code>	<code>a ≤ c</code>	
<code>add_3_7</code>	<code>:</code>	<code>3 + 7 = 10</code>	

Exact?

Local Search

Global Search

- Linear Search
- Indexing
- Example

Overview

Related Tactics

Other ITPs

History

Global Search: Linear Search

<code>exact</code>	<code>le_of_le_of_eq</code>	<code>h₁</code>	<code>(congrArg List.length (id (Eq.symm h₂)))</code>
	<u>Global Search</u> (DiscrTree)	Local Search (SolveByElim)	Local Search (SolveByElim)

What we're proving:

```
example (l1 l2 l3 : List Nat) (h1 : l1.length ≤ l2.length) (h2 : l3 = l2) :  
  l1.length ≤ l3.length := by  
  exact?
```

Our mathlib:

<code>add_2_2</code>	<code>: 2 + 2 = 4</code>	✗
<code>add_2_666</code>	<code>: 2 + 666 = 668</code>	✗
<code>add_3_5</code>	<code>: 3 + 5 = 8</code>	✗
<code>le_of_le_of_eq (h₁ : a ≤ b) (h₂ : b = c)</code>	<code>: a ≤ c</code>	✓
<code>add_3_7</code>	<code>: 3 + 7 = 10</code>	

Global Search: Linear Search

Exact?

Local Search

Global Search

- Linear Search
- Indexing
- Example

Overview

Related Tactics

Other ITPs

History

What we're proving:

```
example (l1 l2 l3 : List Nat) (h1 : l1.length ≤ l2.length) (h2 : l3 = l2) :  
  l1.length ≤ l3.length := by  
  exact?
```

Our mathlib:

add_2_2	: 2 + 2 = 4	✗
add_2_666	: 2 + 666 = 668	✗
add_3_5	: 3 + 5 = 8	✗
le_of_le_of_eq (h1 : a ≤ b) (h2 : b = c)	: a ≤ c	✓
add_3_7	: 3 + 7 = 10	

Fun fact - in Lean 3, `exact?` used linear search!



“`library_search` is just a linear search over the entire environment. We do a tiny bit of filtering, but in the end `apply` is so incredibly fast (in particular, it fails fast) that mostly it is better to just hand everything over to it. I was shocked when I first tried it out that an approach as dumb as that was actually viable.”

Kim Morrison, 2021

Indexing Theorems



Path Indexing



Discrimination Trees

Exact?

Local Search

Global Search

- Linear Search

- Indexing

- Example

Overview

Related Tactics

Other ITPs

History

Term indexing was used in the early days of automated deduction, but it was undocumented or not emphasized in the literature, and the methods are not well known. As a result, it appears that several of the methods were ‘reinvented’. The origins of the two indexing methods remain uncertain. The roots of discrimination-tree indexing appear to lie directly in formula manipulation systems, and the roots of path indexing appear to lie in database technology.

“Experiments with Discrimination-Tree Indexing and Path Indexing”

William McCune, 1990

Path Indexing

Discrimination Tree

Path Indexing

Our mathlib:

Theorem Name	Statement	Prefix Form
<code>add_2_2</code>	$2 + 2 = 4$	$= (+(2, 2), 4)$
<code>add_2_666</code>	$2 + 666 = 668$	$= (+(2, 666), 668)$
<code>add_3_5</code>	$3 + 5 = 8$	$= (+(3, 5), 8)$
<code>add_3_7</code>	$3 + 7 = 10$	$= (+(3, 7), 10)$
<code>le_of_le_of_eq</code>	$a \leq c$	$\leq (*, *)$

Exact?

Local Search

Global Search

- Linear Search

- Indexing

- Example

Overview

Related Tactics

Other ITPs

History

Path Indexing

Discrimination Tree

Path Indexing

Our mathlib:

Theorem Name	Statement	Prefix Form
<code>add_2_2</code>	$2 + 2 = 4$	$= (+ (2, 2), 4)$
<code>add_2_666</code>	$2 + 666 = 668$	$= (+ (2, 666), 668)$
<code>add_3_5</code>	$3 + 5 = 8$	$= (+ (3, 5), 8)$
<code>add_3_7</code>	$3 + 7 = 10$	$= (+ (3, 7), 10)$
<code>le_of_le_of_eq</code>	$a \leq c$	$\leq (*, *)$

Each theorem generates a number of "paths".

Each "path" has the form `[symbol, arg-position, symbol, arg-position, ..., symbol]`.

For example, `add_3_7`: $= (+ (3, 7), 10)$ has five symbols, so it generates five paths:

Symbols (<code>add_3_7</code>)	Path
<code>=</code>	<code>[=]</code>
<code>+</code>	<code>[=, 1, +]</code>
<code>3</code>	<code>[=, 1, +, 1, 3]</code>
<code>7</code>	<code>[=, 1, +, 2, 7]</code>
<code>10</code>	<code>[=, 2, 10]</code>

Exact?

Local Search

Global Search

• Linear Search

• Indexing

• Example

Overview

Related Tactics

Other ITPs

History

Path Indexing

Discrimination Tree

Exact?
Local Search
Global Search
• Linear Search
• Indexing
• Example
Overview
Related Tactics
Other ITPs
History

Per each theorem, per each symbol: compute its path:

Symbols (add_3_7)	Path
=	[=]
+	[=, 1, +]
3	[=, 1, +, 1, 3]
7	[=, 1, +, 2, 7]
10	[=, 2, 10]
Symbols (add_3_5)	Path
=	[=]
+	[=, 1, +]
3	[=, 1, +, 1, 3]
5	[=, 1, +, 2, 5]
8	[=, 2, 8]

And put those paths into a new table:

Path	add_3_5	add_3_7
[=]	✓	✓
[=, 1, +]	✓	✓
[=, 1, +, 1, 3]	✓	✓
[=, 1, +, 2, 5]	✓	
[=, 1, +, 2, 7]		✓
[=, 2, 8]	✓	
[=, 2, 10]		✓

Path Indexing
Discrimination Tree

Path Indexing

- Exact?
- Local Search
- Global Search
 - Linear Search
 - Indexing
 - Example
- Overview
- Related Tactics
- Other ITPs
- History

Path	add_3_5	add_3_7
[=]	✓	✓
[=, 1, +]	✓	✓
[=, 1, +, 1, 3]	✓	✓
[=, 1, +, 2, 5]	✓	
[=, 1, +, 2, 7]		✓
[=, 2, 8]	✓	
[=, 2, 10]		✓

Exact?

Local Search

Global Search

• Linear Search

• Indexing

• Example

Overview

Related Tactics

Other ITPs

History

Path Indexing

Path	add_3_5	add_3_7
[=]	✓	✓
[=, 1, +]	✓	✓
[=, 1, +, 1, 3]	✓	✓
[=, 1, +, 2, 5]	✓	
[=, 1, +, 2, 7]		✓
[=, 2, 8]	✓	
[=, 2, 10]		✓

Suppose our goal is $3 + 7 = 10$.

Compute the path of each symbol of our goal, and intersect the theorem sets:

[=]	$\{\text{add_3_5}, \text{add_3_7}\}$
[=, 1, +]	$\cap \{\text{add_3_5}, \text{add_3_7}\}$
[=, 1, +, 1, 3]	$\cap \{\text{add_3_5}, \text{add_3_7}\}$
[=, 1, +, 2, 7]	$\cap \{\text{add_3_7}\}$
[=, 2, 10]	$\cap \{\text{add_3_7}\}$
	$= \{\text{add_3_7}\}$

Path Indexing

Discrimination Tree

We found our theorem candidate: **add_3_7**.

Indexing Theorems



```
graph TD; A[Indexing Theorems] --> B[Path Indexing]; A --> C[Discrimination Trees];
```

Path Indexing

Discrimination Trees

Two different indexing methods are used: discrimination-tree indexing and path indexing. Some types of term are much better handled by one or the other of the indexing methods. Thus, no clear winner existed between the two methods.

...

The strongest conclusion from the experiments is that discrimination indexing is a clear winner over path indexing for generalization retrieval on the types of term with which I experimented.

“Experiments with Discrimination-Tree Indexing and Path Indexing”
William McCune, 1990

Exact?

Local Search

Global Search

- Linear Search

- Indexing

- Example

Overview

Related Tactics

Other ITPs

History

Path Indexing

Discrimination Tree

Indexing Theorems



Path Indexing

- Z3

Discrimination Trees

- Lean ("Discrimination Tree")
- Rocq ("Discrimination Net")
- Isabelle ("Discrimination Net")
- E Prover (Fingerprint Indexing variant)
- Vampire (Substitution Tree variant)

Two different indexing methods are used: discrimination-tree indexing and path indexing. Some types of term are much better handled by one or the other of the indexing methods. Thus, no clear winner existed between the two methods.

...

The strongest conclusion from the experiments is that discrimination indexing is a clear winner over path indexing for generalization retrieval on the types of term with which I experimented.

"Experiments with Discrimination-Tree Indexing and Path Indexing"
William McCune, 1990

Exact?

Local Search

Global Search

· Linear Search

· Indexing

· Example

Overview

Related Tactics

Other ITPs

History

Path Indexing

Discrimination Tree

Exact?

Local Search

Global Search

- Linear Search
- Indexing
- Example

Overview

Related Tactics

Other ITPs

History

Discrimination Trees

Discrimination tree is a trie

Path Indexing

Discrimination Tree

- Trie
- Lean & Mathlib

Discrimination Trees

Discrimination tree is a trie

Trie nodes is something we do know

Trie leaves is something we WANT to know

Exact?

Local Search

Global Search

- Linear Search

- Indexing

- Example

Overview

Related Tactics

Other ITPs

History

Path Indexing

Discrimination Tree

- Trie

- Lean & Mathlib

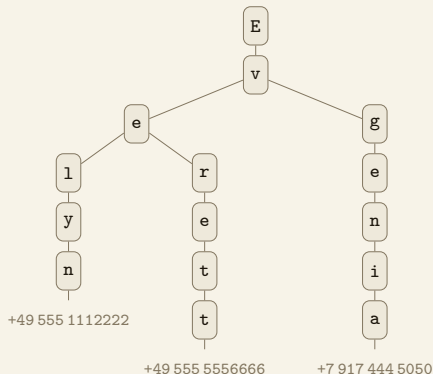
Discrimination Trees

Discrimination tree is a trie

Trie nodes is something we do know

Trie leaves is something we WANT to know

For example,



Exact?

Local Search

Global Search

- Linear Search

- Indexing

- Example

Overview

Related Tactics

Other ITPs

History

Path Indexing

Discrimination Tree

- Trie

- Lean & Mathlib

Discrimination Trees

Our mathlib:

Theorem Name	Statement	Prefix Form
<code>add_2_2</code>	$2 + 2 = 4$	$= (+(2, 2), 4)$
<code>add_2_666</code>	$2 + 666 = 668$	$= (+(2, 666), 668)$
<code>add_3_5</code>	$3 + 5 = 8$	$= (+(3, 5), 8)$
<code>add_3_7</code>	$3 + 7 = 10$	$= (+(3, 7), 10)$
<code>le_of_le_of_eq</code>	$a \leq c$	$\leq (*, *)$

Exact?

Local Search

Global Search

- Linear Search

- Indexing

- Example

Overview

Related Tactics

Other ITPs

History

Path Indexing

Discrimination Tree

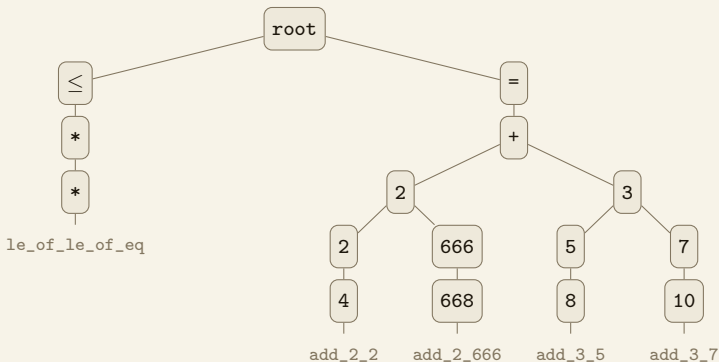
- Trie

- Lean & Mathlib

Discrimination Trees

Our mathlib:

Theorem Name	Statement	Prefix Form
add_2_2	$2 + 2 = 4$	$= (+ (2, 2), 4)$
add_2_666	$2 + 666 = 668$	$= (+ (2, 666), 668)$
add_3_5	$3 + 5 = 8$	$= (+ (3, 5), 8)$
add_3_7	$3 + 7 = 10$	$= (+ (3, 7), 10)$
le_of_le_of_eq	$a \leq c$	$\leq (*, *)$



Exact?

Local Search

Global Search

- Linear Search

- Indexing

- Example

Overview

Related Tactics

Other ITPs

History

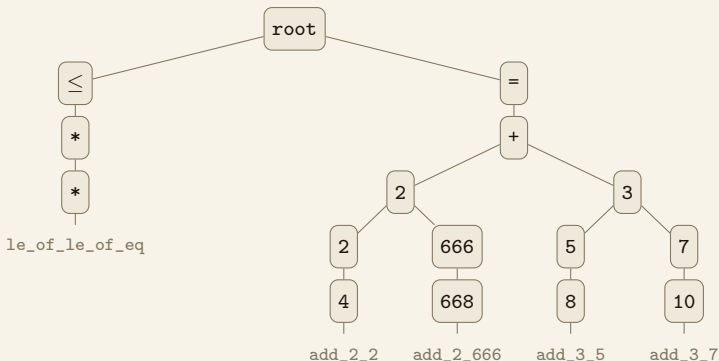
Path Indexing

Discrimination Tree

- Trie

- Lean & Mathlib

Discrimination Trees



This is our discrimination tree, aka `DiscrTree`.

Lean and Mathlib maintain around 15 different `DiscrTree`s. Nodes always contain Lean expressions, leaves contain different stuff.

`DiscrTree α` : nodes are expressions, each leaf is `α`.

`DiscrTree Name` : nodes are expressions, each leaf is a `Name`

`DiscrTree SimpTheorem` : nodes are expressions, each leaf is a `SimpTheorem`

Exact?

Local Search

Global Search

- Linear Search

- Indexing

- Example

Overview

Related Tactics

Other ITPs

History

Path Indexing

Discrimination Tree

- Trie

- Lean & Mathlib

Discrimination Trees: Lean and Mathlib

Exact?

Local Search

Global Search

- Linear Search
- Indexing
- Example

Overview

Related Tactics

Other ITPs

History

Path Indexing

Discrimination Tree

- Trie
- Lean & Mathlib

Used By	What's Stored	DiscrTree α
typeclass instances	@[instance]	DiscrTree InstanceEntry
unification hints	@[unification_hint]	DiscrTree Name
simp	@[simp]	DiscrTree SimpTheorem
simp (simpprocs)	@[simpproc]	DiscrTree SimprocEntry
rfl	@[refl]	DiscrTree Name
symm	@[symm]	DiscrTree Name
ext	@[ext]	DiscrTree ExtTheorem
norm_num	@[norm_num]	DiscrTree NormNumExt
positivity	@[positivity]	DiscrTree PositivityExt
push	@[push]	DiscrTree SimpTheorem
pull	@[pull]	DiscrTree PullTheorem
fun_prop	@[fun_prop]	RefinedDiscrTree GeneralTheorem
exact? / apply?	all imported declarations	LazyDiscrTree (Name $\times$ DeclMod)
rw?	all imported declarations	LazyDiscrTree (Name $\times$ RwDirection)
rw??	all imported declarations	RefinedDiscrTree RewriteLemma

Discrimination Trees: Lean and Mathlib

- Exact?
- Local Search
- Global Search
 - Linear Search
 - Indexing
 - Example
- Overview
- Related Tactics
- Other ITPs
- History

- Path Indexing
- Discrimination Tree
 - Trie
 - Lean & Mathlib

Used By	What's Stored	DiscrTree α
typeclass instances	@[instance]	DiscrTree InstanceEntry
unification hints	@[unification_hint]	DiscrTree Name
simp	@[simp]	DiscrTree SimpTheorem
...		
exact? / apply?	all imported declarations	LazyDiscrTree (Name $\times$ DeclMod)
rw??	all imported declarations	RefinedDiscrTree RewriteLemma

During the compilation of Mathlib, Lean spends approximately 14% of its time running the discrimination trees that allow for efficient association of terms with values.

“Growing Mathlib: maintenance of a large scale mathematical library”
Baanen, Ballard, Commelin, Chen, Rothgang, Testa, 2025

Discrimination Trees: Lean and Mathlib

Used By	What's Stored	DiscrTree α
typeclass instances	@[instance]	DiscrTree InstanceEntry
unification hints	@[unification_hint]	DiscrTree Name
simp	@[simp]	DiscrTree SimpTheorem
...		
exact? / apply?	all imported declarations	LazyDiscrTree (Name $\times$ DeclMod)
rw??	all imported declarations	RefinedDiscrTree RewriteLemma

During the compilation of Mathlib, Lean spends approximately 14% of its time running the discrimination trees that allow for efficient association of terms with values.

“Growing Mathlib: maintenance of a large scale mathematical library”
Baanen, Ballard, Commelin, Chen, Rothgang, Testa, 2025

A single large discrimination tree?

- Path Indexing
- Discrimination Tree
 - Trie
 - Lean & Mathlib

Discrimination Trees: Lean and Mathlib

- Exact?
- Local Search
- Global Search
 - Linear Search
 - Indexing
 - Example
- Overview
- Related Tactics
- Other ITPs
- History

- Path Indexing
- Discrimination Tree
 - Trie
 - Lean & Mathlib

Used By	What's Stored	DiscrTree α
typeclass instances	@[instance]	DiscrTree InstanceEntry
unification hints	@[unification_hint]	DiscrTree Name
simp	@[simp]	DiscrTree SimpTheorem
...		
exact? / apply?	all imported declarations	LazyDiscrTree (Name $\times$ DeclMod)
rw??	all imported declarations	RefinedDiscrTree RewriteLemma

During the compilation of Mathlib, Lean spends approximately 14% of its time running the discrimination trees that allow for efficient association of terms with values.

“Growing Mathlib: maintenance of a large scale mathematical library”
Baanen, Ballard, Commelin, Chen, Rothgang, Testa, 2025

A single large discrimination tree?

Discrimination Trees: Lean and Mathlib

Used By	What's Stored	DiscrTree α
typeclass instances	@[instance]	DiscrTree InstanceEntry
unification hints	@[unification_hint]	DiscrTree Name
simp	@[simp]	DiscrTree SimpTheorem
...		
exact? / apply?	all imported declarations	LazyDiscrTree (Name $\times$ DeclMod)
rw??	all imported declarations	RefinedDiscrTree RewriteLemma

During the compilation of Mathlib, Lean spends approximately 14% of its time running the discrimination trees that allow for efficient association of terms with values.

“Growing Mathlib: maintenance of a large scale mathematical library”
Baanen, Ballard, Commelin, Chen, Rothgang, Testa, 2025

A single large discrimination tree?

- Leaves store different stuff (e.g. InstanceEntry vs Name) - not a problem

Exact?

Local Search

Global Search

- Linear Search
- Indexing
- Example

Overview

Related Tactics

Other ITPs

History

Path Indexing

Discrimination Tree

- Trie
- Lean & Mathlib

Discrimination Trees: Lean and Mathlib

Used By	What's Stored	DiscrTree α
typeclass instances	@[instance]	DiscrTree InstanceEntry
unification hints	@[unification_hint]	DiscrTree Name
simp	@[simp]	DiscrTree SimpTheorem
...		
exact? / apply?	all imported declarations	LazyDiscrTree (Name $\times$ DeclMod)
rw??	all imported declarations	RefinedDiscrTree RewriteLemma

During the compilation of Mathlib, Lean spends approximately 14% of its time running the discrimination trees that allow for efficient association of terms with values.

“Growing Mathlib: maintenance of a large scale mathematical library”
Baanen, Ballard, Commelin, Chen, Rothgang, Testa, 2025

A single large discrimination tree?

- ▶ Leaves store different stuff (e.g. InstanceEntry vs Name) - not a problem
- ▶ Different theorems are processed (e.g. @[simp] vs @[norm_num]) - not a problem

Exact?

Local Search

Global Search

- Linear Search
- Indexing
- Example

Overview

Related Tactics

Other ITPs

History

Path Indexing

Discrimination Tree

- Trie
- Lean & Mathlib

Discrimination Trees: Lean and Mathlib

Used By	What's Stored	DiscrTree α
typeclass instances	@[instance]	DiscrTree InstanceEntry
unification hints	@[unification_hint]	DiscrTree Name
simp	@[simp]	DiscrTree SimpTheorem
...		
exact? / apply?	all imported declarations	LazyDiscrTree (Name $\times$ DeclMod)
rw??	all imported declarations	RefinedDiscrTree RewriteLemma

During the compilation of Mathlib, Lean spends approximately 14% of its time running the discrimination trees that allow for efficient association of terms with values.

“Growing Mathlib: maintenance of a large scale mathematical library”
Baanen, Ballard, Commelin, Chen, Rothgang, Testa, 2025

A single large discrimination tree?

- ▶ Leaves store different stuff (e.g. InstanceEntry vs Name) - not a problem
- ▶ Different theorems are processed (e.g. @[simp] vs @[norm_num]) - not a problem
- ▶ Actual reason: **nodes** store different stuff (e.g. exact? full expression, simp LHS)

Exact?

Local Search

Global Search

- Linear Search
- Indexing
- Example

Overview

Related Tactics

Other ITPs

History

Path Indexing

Discrimination Tree

- Trie
- Lean & Mathlib

Exact?

Local Search

Global Search

- Linear Search
- Indexing
- Example

Overview

Related Tactics

Other ITPs

History

Global Search: Example

Our Mathlib:

```
theorem Nat.add_comm :  $\forall (n\ m : \text{Nat}), n + m = m + n$   
theorem Nat.zero_le :  $\forall (n : \text{Nat}), 0 \leq n$   
theorem Nat.le_refl :  $\forall (n : \text{Nat}), n \leq n$   
theorem Nat.le_succ :  $\forall (n : \text{Nat}), n \leq n.\text{succ}$ 
```

Our goal:

```
⊢ 0 ≤ 5
```

We call `exact?`, and the global search among all Mathlib lemmas starts:

- ▶ We convert each Mathlib theorem into `[Key, Key, Key]`, and put all theorems into a **DiscrTree**
- ▶ We convert our goal into `[Key, Key, Key]`
- ▶ We match the resulting **DiscrTree** to our goal!

Global Search: What We Do To Mathlib

```
theorem Nat.add_comm :  $\forall (n\ m : \text{Nat}),\ n + m = m + n$ 
```

Exact?

Local Search

Global Search

- Linear Search

- Indexing

- Example

Overview

Related Tactics

Other ITPs

History

Exact?

Local Search

Global Search

- Linear Search

- Indexing

- Example

Overview

Related Tactics

Other ITPs

History

Global Search: What We Do To Mathlib

```
theorem Nat.add_comm :  $\forall (n\ m : \text{Nat}),\ n + m = m + n$ 
```

```
theorem Nat.add_comm :  $\forall (n\ m : \text{Nat}),\ \text{Eq Nat } (\text{Nat.add } n\ m)\ (\text{Nat.add } m\ n)$ 
```

Global Search: What We Do To Mathlib

Exact?

Local Search

Global Search

- Linear Search
- Indexing
- Example

Overview

Related Tactics

Other ITPs

History

```
theorem Nat.add_comm : ∀ (n m : Nat), n + m = m + n
theorem Nat.add_comm : ∀ (n m : Nat), Eq Nat (Nat.add n m) (Nat.add m n)
theorem Nat.add_comm : .mkAppN (.const 'Eq) #[.const 'Nat, .mkAppN (.const
'Nat.add) #[.fvar 'n, .fvar 'm], .mkAppN (.const 'Nat.add) #[.fvar 'm, .fvar
'n]]
```

```
inductive Expr
| bvar      : Nat
| fvar      : FVarId
| mvar      : MVarId
| sort      : Level
| const     : Name → List Level
| app       : Expr → Expr
| lam       : Name → Expr → Expr
| forallE   : Name → Expr → Expr
| letE      : Name → ...
| lit       : Literal
| mdata     : MData → Expr
| proj      : Name → Nat → Expr
```


Global Search: What We Do To Mathlib

Exact?

Local Search

Global Search

- Linear Search
- Indexing
- Example

Overview

Related Tactics

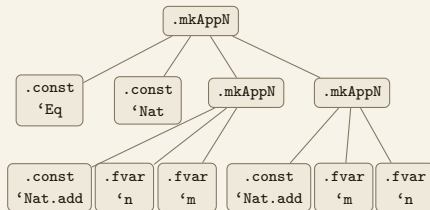
Other ITPs

History

```
theorem Nat.add_comm :  $\forall (n m : \text{Nat}), n + m = m + n$   
theorem Nat.add_comm :  $\forall (n m : \text{Nat}), \text{Eq Nat } (\text{Nat.add } n \ m) \ (\text{Nat.add } m \ n)$   
theorem Nat.add_comm : .mkAppN (.const 'Eq) #[.const 'Nat, .mkAppN (.const  
'Nat.add) #[.fvar 'n, .fvar 'm], .mkAppN (.const 'Nat.add) #[.fvar 'm, .fvar  
'n]]
```

inductive Expr

```
| bvar    : Nat  
| fvar    : FVarId  
| mvar    : MVarId  
| sort    : Level  
| const   : Name → List Level  
| app     : Expr → Expr  
| lam     : Name → Expr → Expr  
| forallE : Name → Expr → Expr  
| letE    : Name → ...  
| lit     : Literal  
| mdata   : MData → Expr  
| proj    : Name → Nat → Expr
```



Global Search: What We Do To Mathlib

Exact?

Local Search

Global Search

- Linear Search
- Indexing
- Example

Overview

Related Tactics

Other ITPs

History

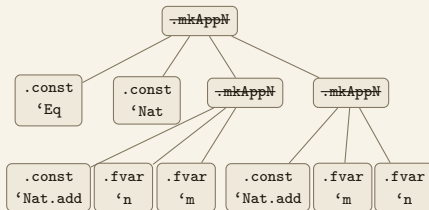
```
theorem Nat.add_comm :  $\forall (n m : \text{Nat}), n + m = m + n$ 
```

```
theorem Nat.add_comm :  $\forall (n m : \text{Nat}), \text{Eq Nat } (\text{Nat.add } n \ m) \ (\text{Nat.add } m \ n)$ 
```

```
theorem Nat.add_comm : .mkAppN (.const 'Eq) #[.const 'Nat, .mkAppN (.const  
'Nat.add) #[.fvar 'n, .fvar 'm], .mkAppN (.const 'Nat.add) #[.fvar 'm, .fvar  
'n]]
```

inductive Expr

```
| bvar   : Nat  
| fvar   : FVarId  
| mvar   : MVarId  
| sort   : Level  
| const  : Name → List Level  
| app    : Expr → Expr  
| lam    : Name → Expr → Expr  
| forallE : Name → Expr → Expr  
| letE   : Name → ...  
| lit    : Literal  
| mdata  : MData → Expr  
| proj   : Name → Nat → Expr
```



```
theorem Nat.add_comm : [.const 'Eq 3, .const 'Nat 0, .const 'Nat.add 2, .fvar  
'n 0, .fvar 'm 0, .const 'Nat.add 2, .fvar 'm 0, .fvar 'n 0]
```

Global Search: What We Do To Mathlib

Exact?

Local Search

Global Search

- Linear Search
- Indexing
- Example

Overview

Related Tactics

Other ITPs

History

```
theorem Nat.add_comm : [Expr.const 'Eq 3, Expr.const 'Nat 0, Expr.const  
'Nat.add 2, Expr.fvar 'n 0, Expr.fvar 'm 0, Expr.const 'Nat.add 2, Expr.fvar  
'm 0, Expr.fvar 'n 0]
```

inductive Expr

```
| const : Name → List Level  
| lit : Literal  
| proj : Name → Nat → Expr  
| forallE : Name → Expr → Expr → BinderInfo  
| fvar : FVarId  
| bvar : Nat  
| mvar : MVarId  
| sort : Level  
| mdata : MData → Expr  
| app : Expr → Expr  
| lam : Name → Expr → Expr → BinderInfo  
| letE : Name → Expr → Expr → Expr → Bool
```

inductive Key

```
| const : Name → Nat  
| lit : Literal  
| proj : Name → Nat → Nat  
| arrow  
| fvar : FVarId → Nat  
| *  
| other
```

```
theorem Nat.add_comm : [Key.const 'Eq 3, Key.const 'Nat 0, Key.const 'Nat.add 2,  
*, *, Key.const 'Nat.add 2, *, *]
```

Global Search: What We Do To Mathlib

BEFORE:

```
theorem Nat.add_comm :  $\forall (n\ m : \text{Nat}), n + m = m + n$ 
```

STEPS:

1. We have a Mathlib theorem

```
theorem Nat.add_comm (n m : Nat) :  $n + m = m + n$ 
```

2. Reduce the expression, strip forall

```
theorem Nat.add_comm :  $\forall (n\ m : \text{Nat}), \text{Eq Nat } (\text{Nat.add } n\ m) (\text{Nat.add } m\ n)$ 
```

3. Turn Expr tree into a list of Exprs

```
theorem Nat.add_comm : [Expr.const 'Eq 3, Expr.const 'Nat 0, Expr.const  
'Nat.add 2, Expr.fvar 'n 0, Expr.fvar 'm 0, Expr.const 'Nat.add 2,  
Expr.fvar 'm 0, Expr.fvar 'n 0]
```

4. Map a list of Exprs to a list of Keys

```
theorem Nat.add_comm : [Key.const 'Eq 3, Key.const 'Nat 0,  
Key.const 'Nat.add 2, *, *, Key.const 'Nat.add 2, *, *]
```

AFTER:

```
theorem Nat.add_comm : [Key.const 'Eq 3, Key.const 'Nat 0, Key.const 'Nat.add 2,  
*, *, Key.const 'Nat.add 2, *, *]
```

Exact?

Local Search

Global Search

- Linear Search

- Indexing

- Example

Overview

Related Tactics

Other ITPs

History

Exact?

Local Search

Global Search

- Linear Search

- Indexing

- Example

Overview

Related Tactics

Other ITPs

History

Global Search: What We Do To Our Goal

BEFORE:

$\vdash 0 \leq 5$

STEPS:

1. We have a goal

$\vdash 0 \leq 5$

2. Reduce the expression, strips forall's

$\vdash \text{Nat.le } 0 \ 5$

3. Turn Expr tree into a list of Exprs

$\vdash [\text{Expr.const 'Nat.le } 2, \text{Expr.lit } 0, \text{Expr.lit } 5]$

4. Map a list of Exprs to a list of Keys

$\vdash [\text{Key.const 'Nat.le } 2, \text{Key.lit } 0, \text{Key.lit } 5]$

AFTER:

$\vdash [\text{Key.const 'Nat.le } 2, \text{Key.lit } 0, \text{Key.lit } 5]$

Exact?

Local Search

Global Search

- Linear Search
- Indexing
- Example

Overview

Related Tactics

Other ITPs

History

Global search: DiscrTree

All Mathlib lemmas:

```
theorem Nat.add_comm : [Key.const 'Eq 3, Key.const 'Nat 0, Key.const 'Nat.add 2,  
                        *, *, Key.const 'Nat.add 2, *, *]  
theorem Nat.zero_le  : [Key.const 'Nat.le 2, Key.lit 0, *]  
theorem Nat.le_refl  : [Key.const 'Nat.le 2, *, *]  
theorem Nat.le_succ  : [Key.const 'Nat.le 2, *, *]
```

Our goal:

```
⊢ [Key.const 'Nat.le 2, Key.lit 0, Key.lit 5]
```

Global Search: DiscrTree

Exact?

Local Search

Global Search

- Linear Search

- Indexing

- Example

Overview

Related Tactics

Other ITPs

History

```
theorem Nat.add_comm :
```

```
theorem Nat.zero_le  :
```

```
theorem Nat.le_refl  :
```

```
theorem Nat.le_succ  :
```

Global Search: DiscrTree

Exact?

Local Search

Global Search

- Linear Search
- Indexing
- Example

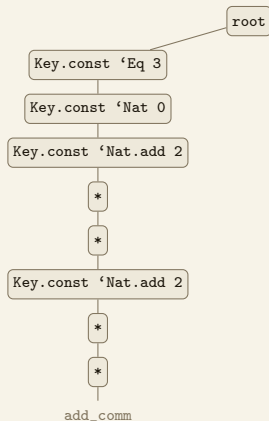
Overview

Related Tactics

Other ITPs

History

```
theorem Nat.add_comm : [Key.const 'Eq 3, Key.const 'Nat 0, Key.const 'Nat.add 2,  
                        *, *, Key.const 'Nat.add 2, *, *]  
  
theorem Nat.zero_le :  
  
theorem Nat.le_refl :  
  
theorem Nat.le_succ :
```



Global Search: DiscrTree

Exact?

Local Search

Global Search

- Linear Search
- Indexing
- Example

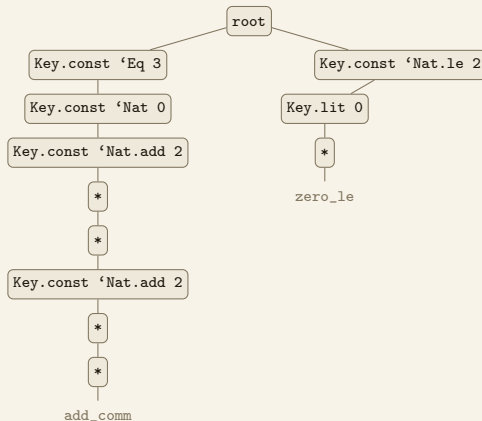
Overview

Related Tactics

Other ITPs

History

```
theorem Nat.add_comm : [Key.const 'Eq 3, Key.const 'Nat 0, Key.const 'Nat.add 2,  
                        *, *, Key.const 'Nat.add 2, *, *]  
theorem Nat.zero_le : [Key.const 'Nat.le 2, Key.lit 0, *]  
theorem Nat.le_refl :  
theorem Nat.le_succ :
```



Global Search: DiscrTree

Exact?

Local Search

Global Search

- Linear Search
- Indexing
- Example

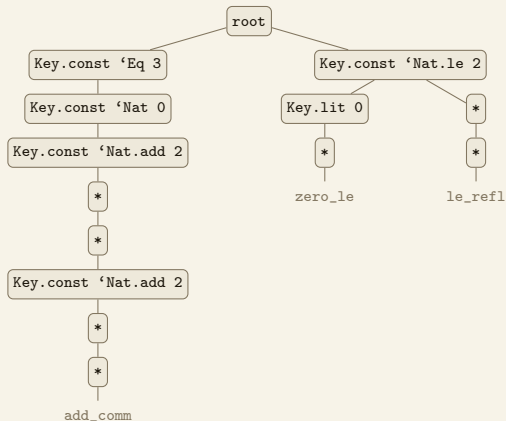
Overview

Related Tactics

Other ITPs

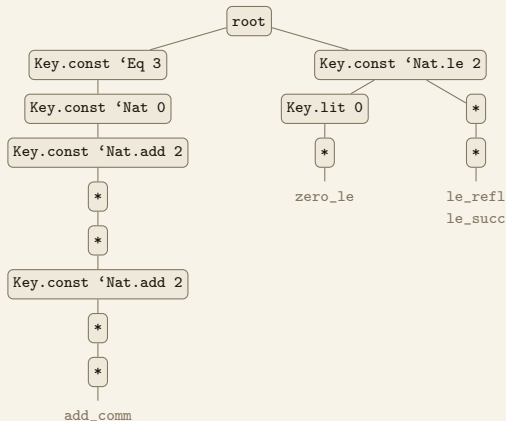
History

```
theorem Nat.add_comm : [Key.const 'Eq 3, Key.const 'Nat 0, Key.const 'Nat.add 2,  
                        *, *, Key.const 'Nat.add 2, *, *]  
theorem Nat.zero_le  : [Key.const 'Nat.le 2, Key.lit 0, *]  
theorem Nat.le_refl  : [Key.const 'Nat.le 2, *, *]  
theorem Nat.le_succ  :
```



Global Search: DiscrTree

```
theorem Nat.add_comm : [Key.const 'Eq 3, Key.const 'Nat 0, Key.const 'Nat.add 2,
                        *, *, Key.const 'Nat.add 2, *, *]
theorem Nat.zero_le   : [Key.const 'Nat.le 2, Key.lit 0, *]
theorem Nat.le_refl   : [Key.const 'Nat.le 2, *, *]
theorem Nat.le_succ   : [Key.const 'Nat.le 2, *, *]
```



Global Search: Mathlib + Our Goal

All Mathlib lemmas:

```
theorem Nat.add_comm : [Key.const 'Eq 3, Key.const 'Nat 0, Key.const 'Nat.add 2,  
                        *, *, Key.const 'Nat.add 2, *, *]  
theorem Nat.zero_le  : [Key.const 'Nat.le 2, Key.lit 0, *]  
theorem Nat.le_refl  : [Key.const 'Nat.le 2, *, *]  
theorem Nat.le_succ  : [Key.const 'Nat.le 2, *, *]
```

Our goal:

```
⊢ [Key.const 'Nat.le 2, Key.lit 0, Key.lit 5]
```

Exact?

Local Search

Global Search

- Linear Search
- Indexing
- Example

Overview

Related Tactics

Other ITPs

History

Exact?

Local Search

Global Search

- Linear Search

- Indexing

- Example

Overview

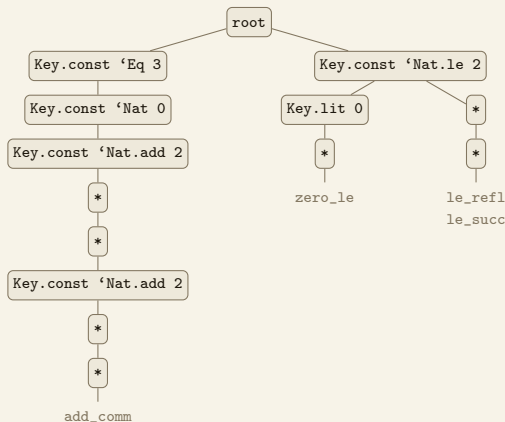
Related Tactics

Other ITPs

History

Global Search: Mathlib + Our Goal

All Mathlib lemmas:



Our goal:

$\vdash$ [Key.const 'Nat.le 2, Key.lit 0, Key.lit 5]

Exact?

Local Search

Global Search

- Linear Search

- Indexing

- Example

Overview

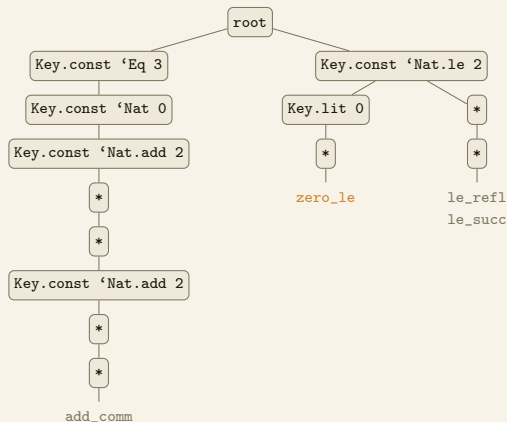
Related Tactics

Other ITPs

History

Global Search: Mathlib + Our Goal

All Mathlib lemmas:



Our goal:

$\vdash$ [Key.const 'Nat.le 2, Key.lit 0, Key.lit 5]

Exact?

Local Search

Global Search

- Linear Search

- Indexing

- Example

Overview

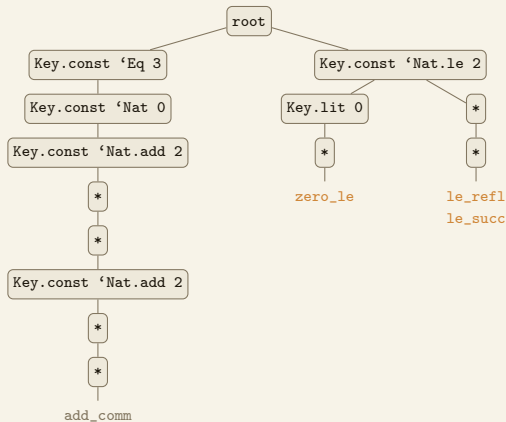
Related Tactics

Other ITPs

History

Global Search: Mathlib + Our Goal

All Mathlib lemmas:



Our goal:

$\vdash$ [Key.const 'Nat.le 2, Key.lit 0, Key.lit 5]

Exact?

Local Search

Global Search

- Linear Search

- Indexing

- Example

Overview

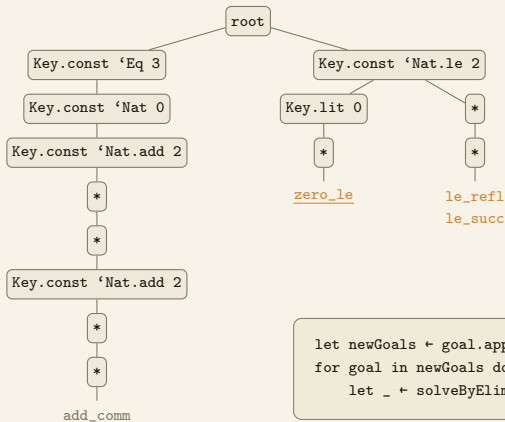
Related Tactics

Other ITPs

History

Global Search: Mathlib + Our Goal

All Mathlib lemmas:



```
let newGoals ← goal.apply zero_le
for goal in newGoals do
  let _ ← solveByElim goal
```

Our goal:

```
⊢ [Key.const 'Nat.le 2, Key.lit 0, Key.lit 5]
```


Exact?

Local Search

Global Search

- Linear Search

- Indexing

- Example

Overview

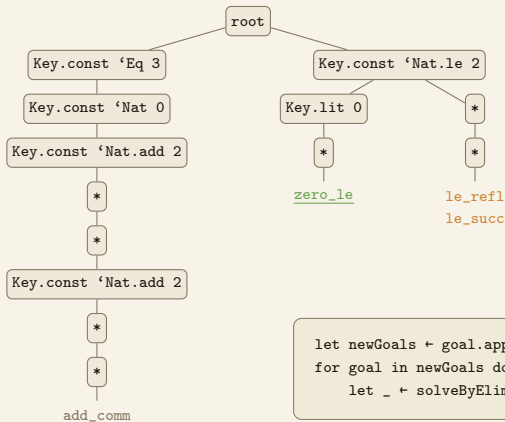
Related Tactics

Other ITPs

History

Global Search: Mathlib + Our Goal

All Mathlib lemmas:



```
let newGoals ← goal.apply zero_le
for goal in newGoals do
  let _ ← solveByElim goal
```

Our goal:

```
⊢ [Key.const 'Nat.le 2, Key.lit 0, Key.lit 5]
```

Overview

Exact?

Local Search

Global Search

Overview

Related Tactics

Other ITPs

History

```
example (l1 l2 l3 : List Nat) (h1 : l1.length ≤ l2.length) (h2 :  
l3 = l2) : l1.length ≤ l3.length := by  
  exact?
```

```
example (l1 l2 l3 : List Nat) (h1 : l1.length ≤ l2.length) (h2 :  
l3 = l2) : l1.length ≤ l3.length := by  
  exact le_of_le_of_eq h1 (congrArg List.length (id (Eq.symm h2)))
```

```
exact  le_of_le_of_eq      h1  (congrArg List.length (id (Eq.symm h2)))
```

	Global Search	Local Search	Local Search
	(DiscrTree)	(SolveByElim)	(SolveByElim)

Exact?

Local Search

Global Search

Overview

Related Tactics

Other ITPs

History

Overview

```
example (l1 l2 l3 : List Nat) (h1 : l1.length ≤ l2.length) (h2 :  
l3 = l2) : l1.length ≤ l3.length := by  
  exact?
```

```
example (l1 l2 l3 : List Nat) (h1 : l1.length ≤ l2.length) (h2 :  
l3 = l2) : l1.length ≤ l3.length := by  
  exact le_of_le_of_eq h1 (congrArg List.length (id (Eq.symm h2)))
```

exact	<u>le_of_le_of_eq</u>	<u>h₁</u>	<u>(congrArg List.length (id (Eq.symm h₂)))</u>
	Global Search (DiscrTree)	Local Search (SolveByElim)	Local Search (SolveByElim)

Demo: Real Project

Exact?

Local Search

Global Search

Overview

Related Tactics

- Apply?
- Rw?
- Rw??

Other ITPs

History

Related Tactics

Related Tactics: Apply?

`apply?` and `exact?` call the same function (*/Lean/Elab/Tactic/LibrarySearch.lean*)

- `exact?` calls `exact? ref config required requireClose := True`
- `apply?` calls `exact? ref config required requireClose := False`

```
let newGoals ← goal.apply lemma that matched our goal in DiscrTree
-- lemma is guaranteed to be returned whatever happens to the remaining subgoals
for goal in newGoals do
  let _ ← solveByElim goal
```

Exact?

Local Search

Global Search

Overview

Related Tactics

- Apply?
- Rw?
- Rw??

Other ITPs

History

Exact?

Local Search

Global Search

Overview

Related Tactics

• Apply?

• Rw?

• Rw??

Other ITPs

History

Related Tactics: Rw?

- Builds the `DiscrTree` **only** from theorems of the form `x = y` or `x ↔ y`
- Puts **both sides** into the `DiscrTree`:

`x` 's `[Key, Key, Key]` $\rightarrow$ `DiscrTree` (for `rw [lemma]`)

`y` 's `[Key, Key, Key]` $\rightarrow$ `DiscrTree` (for `rw [← lemma]`)

- Subdivides our goal into **all subexpressions**,
and matches all of them to the `DiscrTree` one by one

Exact?

Local Search

Global Search

Overview

Related Tactics

- Apply?
- Rw?
- Rw??

Other ITPs

History

Related Tactics: Rw??

A **vscode widget** by Jovan Gerbscheid.

You write `rw??`, then **shift-click** any subexpression in the goal.
The widget shows all rewrites for that exact expression.

Uses a **RefinedDiscrTree**.

Exact?

Local Search

Global Search

Overview

Related Tactics

Other ITPs

History

Other Proof Assistants

	Search Engine	exact?	rw?
Lean	<code>#loogle</code> (command)	<code>exact?</code> / <code>apply?</code>	<code>rw?</code> / <code>rw??</code>
Rocq	<code>Search</code> (command)	<code>auto</code> (production tactic!) ▶ similar to our <code>solve_by_elim</code>, except does consider nonlocal lemmas ▶ we specify a "hint db" (doesn't index whole library) ▶ backtracking with depth 5	—
Isabelle	<code>find_theorems</code> (command)	<code>solve_direct</code> ▶ similar to our implementation ▶ but uses linear search (even though isabelle does have discr trees elsewhere) ▶ and uses the equivalent of our <code>assumption</code> to discharge the goals	—

History



- Exact?
 - Local Search
 - Global Search
 - Overview
 - Related Tactics
 - Other ITPs
 - History
- # Questions